Arithmetic Expression Evaluator

User's Manual
Version 1.0

Winston Industries LLC

Revision History

Date	Version	Description	Author
10/May/2026	1.0	Updated User Manual	Caden Gudenkaufd

Table of Contents

User's Manual

1. Introduction

The Arithmetic Expression Evaluator (AEE) is a command line program written in C++ that lets you type in a math expression and instantly get the answer using multiple operator types. It handles everything from basic addition to nested parentheses and exponents, following the standard PEMDAS order of operations automatically.

If your expression has a mistake in it, the program will not crash. Instead, it prints a clear error message telling you exactly what went wrong.

Features:

1. Supports six operators: +, -, *, /, %, and **

2. Follows PEMDAS order of operations automatically

3. Handles negative numbers and decimal results

4. Supports parentheses for grouping

5. Clear error messages for invalid expressions

6. Built in unit tests to verify the program is working correctly

Team:

Name	Role	Email
Davina Love	Project Leader	dlove13@ku.edu
Landrie Rolla	Assistant Project Leader	landrierolla@ku.edu
Liam Gunther	Technical Leader	lgunther@ku.edu
Christian Mitchell	Team Administrator	christian.mitchell@ku.edu
Conner Magee	Data Administrator / QA Engineer	connermagee@ku.edu
Caden Gudenkauf	Assistant Team Administrator	caden.gudenkauf@ku.edu

2. Getting Started

Before running the program, make sure these two tools are installed on your computer:

1. g++ - the C++ compiler that builds the program. Requires C++17 or newer.
2. make - the build tool that compiles everything for you with one command.

To check if you have them, open a terminal and type g++ --version and press Enter. If you see a version number, you are good. Do the same for make --version.

3. Building the Program

Once you have the tools ready, follow these steps:

1. Open a terminal (Command Prompt, PowerShell, or a Linux/Mac terminal).
2. Navigate to the src folder inside the project:

```
cd path/to/Software-Engineering-Project-2026/src
```

3. Run the build command:

```
make
```

You will see a few lines of compiler output. When it finishes without errors, a file called main will appear in the src folder.

If you ever want to rebuild from scratch, run make clean first, then make again.

4. Running the Program

From inside the src folder, run:

```
./main          (Mac / Linux)
main.exe        (Windows)
```

You will see a simple menu:

```
1. Evaluate an expression
2. Run unit tests
(any other key) Exit
```

5. Entering an Expression

Select option 1 and type your expression when prompted, then press Enter. For example:

```
Enter expression: (3 + 5) * 2
Result: 16
```

Spaces between numbers and operators are optional but recommended for readability.

Supported operators:

Operator	Name	Example	Result
+	Addition	3 + 4	7
−	Subtraction	10 - 6	4
*	Multiplication	3 * 4	12

/	Division	7 / 2	3.5
%	Modulo	10 % 3	1
**	Exponentiation	2 ** 8	256
( )	Parentheses	(1 + 2) * 4	12

6. Interpreting the Result

The result is always printed as a decimal number. For example, 7 / 2 gives 3.5, not 3. If the expression has a problem, you will see an error message instead. See Section 4 for a full list of errors and what they mean.

7. Operator Precedence

The program follows PEMDAS automatically, so you do not need extra parentheses unless you want to change the default order. Here is how operators rank:

Operator(s)	Precedence	Associativity
+ -	Lowest (1)	Left to right
* / %	Middle (2)	Left to right
**	Highest (3)	Right to left

Left to right means 10 - 3 - 2 is calculated as (10 - 3) - 2 = 5. Right to left (only for **) means 2 ** 3 ** 2 is calculated as 2 ** (3 ** 2) = 512.

8. Negative Numbers

Negative numbers are supported anywhere in an expression. Just write the minus sign directly before the number, for example:

```
5 * -3          result: -15
(14 + 16) / -4  result: -7.5
```

9. Nested Parentheses

You can nest parentheses as deep as you need. The program evaluates the innermost ones first:

```
((2 + 3) * (4 - 1)) ** 2     result: 225
```

10. Running the Built-in Tests

Select option 2 from the main menu to run the test suite. It checks three things:

7. Tokenizer Tests - checks that the program correctly reads numbers, operators, and parentheses from an expression.

8. Parser Tests - checks that expressions are converted to the right evaluation order and that errors are caught properly.

9. Evaluator Tests - checks that the correct numeric result is produced.

Each test prints PASSED or FAILED. A fully working build should show PASSED for every test.

11. Note on Saving / Variables

This version of the program does not support saving expressions, loading previous results, or defining custom variables and functions. Each expression is evaluated on its own.

12. Troubleshooting

Problem	**Fix**
g++ or make says "not found"	Install the compiler. Windows: use MinGW or WSL. Mac: run xcode-select --install. Linux: run sudo apt install build-essential.
Program closes immediately on Windows	Do not double-click the .exe file. Open a terminal (Command Prompt or PowerShell), navigate to the src folder, and run main.exe from there.
Compile error after editing source files	Run make clean, then run make again to rebuild from scratch.
Expression looks correct but gives an error	Check the error message printed. The most common issues are a missing parenthesis, two operators in a row, or starting the expression with an operator.
All unit tests fail	Make sure you are inside the src folder and that you ran make before launching the program.

13. Error Messages

When something is wrong with your expression, the program prints one of these errors:

Error	**What it means**	**Example**	**Fix**
`Mismatched parenthesis`	An opening or closing parenthesis has no match.	(3 + 2	Add the missing)
`Adjacent operators`	Two operators appear next to each other.	14 + + 3	Remove the extra operator
`Invalid syntax`	An operator sits right next to a parenthesis incorrectly.	(/ 1	Add a number before the operator
`Invalid expression`	The expression starts with an operator.	+ 5	Put a number before the operator
`Divide by zero`	The divisor is zero.	10 / 0	Change the divisor to a non-zero value

14. Examples

Below are a variety of expressions you can try to get familiar with the program.

Expression	Result	What it shows
`3 + 4`	7	Basic addition
`10 - 3 - 2`	5	Left-to-right subtraction
`2 ** 3 ** 2`	512	Right-to-left exponentiation
`(1 + 2) * 3`	9	Parentheses changing order of operations
`7 / 2`	3.5	Division returning a decimal
`10 % 3`	1	Modulo (remainder after division)
`5 * -3`	-15	Using a negative number
`((2 + 3) * (4 - 1)) ** 2`	225	Nested parentheses
`(14 + 16 - 12) * 3 / -4 % 6 ** 2`	-4.5	Complex mixed expression

15. Glossary of Terms

Term	Definition
Operator	A symbol that performs a calculation, such as + or *.
Operand	A number that an operator acts on. In 3 + 4, both 3 and 4 are operands.
Expression	A combination of numbers, operators, and parentheses that evaluates to a single value.
PEMDAS	Standard order of operations: Parentheses, Exponents, Multiplication/Division, Addition/Subtraction.
Precedence	The priority of an operator. Higher precedence operators are applied first.
Associativity	The direction operators of equal precedence are evaluated. Most go left to right; ** goes right to left.
Modulo (%)	Returns the remainder after division. 10 % 3 = 1 because 10 / 3 = 3 remainder 1.
Exponentiation (**)	Raises a number to a power. 2 ** 8 means 2 to the power of 8 = 256.
Double	A decimal number. The program always returns results as doubles.
RPN	Reverse Polish Notation. An internal format used to evaluate expressions.
Compiler	A tool that converts source code into a runnable program. This project uses g++.

16. FAQ

Can I use ^ for exponentiation?

No. Use ** instead. Typing ^ will give an invalid expression error.

Does the program save my previous expressions?

Each expression is evaluated independently with no history/save..

Can I define variables or custom functions?

No. This version only supports expressions made up of numbers, operators, and parentheses.

Why does 7 / 2 give 3.5 instead of 3?

The program always uses decimal arithmetic, so division returns the exact result. If you want to check the remainder, use %. For example 7 % 2 gives 1.

Do spaces matter?

Spaces are optional. (3+5)*2 and (3 + 5) * 2 both work. Spaces just make expressions easier to read.